
CprE 3810

Homework 01

Note: All students are supposed to turn in the source files (.c for C code and .s/.asm for assembly files), verified and tested in RARS to be graded.

1. ISAs

- (a) Identify the (concrete) benefits and drawbacks (at least one of each and three total) of establishing a hardware abstraction that uses simple, primitive instructions rather than implementing a high-level programming language directly.
- (b) RISC-V: an example of ISA design decisions. Using the specifications for the ISA (https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view?usp=drive_link) and ABI (https://drive.google.com/file/d/1Ja_Tpp_5Me583CGVD-BIZMlgGBn1KU4R/view?usp=drive_link), answer the following questions and cite the page(s) in the spec you used to answer the question:
 - i. What is the XLEN for RV32I?
 - ii. How many general purpose integer registers are there for RV32I? Why 32 and not 8 or 16? Why not 64? I.e., what are the trade-offs of changing the number of architectural registers?
 - iii. Register x0 has a special use. What is it? Is this enforced by a hardware implementation (i.e., microarchitecture), the ISA, the ABI, or the assembler?
 - iv. The PC is a register in the architectural state. What is the purpose of the PC? Can you use the `add` instruction to modify it?
 - v. Does a HW implementation of RV32I need to identify overflow for any of the base instructions? Why not?
 - vi. Priming for a later callback: Why were branches with integrated arithmetic comparisons included in the ISA?

2. RISC-V

- (a) Identify three products not mentioned in lecture videos that use a RISC-V-based processor. Cite the reference and why you believe it.
- (b) RISC-V does not have a `clear` instruction (`clr dst`). The intent of this operation is to set the `dst` register to 0. Why would such a simple, basic instruction not be included in RISC-V? List three arithmetic or logical instructions or instruction sequences not mentioned during lecture that produce the same effect as a `clr` instruction.
- (c) What does the following program do (give a description in English sentences)?
`xor x1, x1, x2 xor x2, x1, x2 xor x1, x1, x2`
- (d) Assume that `int` variables `a`, `b`, `c`, and `d` are mapped to registers `s0`, `s1`, `s2`, and `s3`, respectively. Write a RISC-V program that implements: `a = (((b-7) / 2) & c) > (d | 0xF00)`; Use only `add`, `sub`, `addi`, `and`, `andi`, `or`, `ori`, `sll`, `slli`, `srl`, `srli`, `sra`, `srai`, `slt`, and `slti` instructions. Since `b`, `c`, and `d` must not be overwritten, use as many temporary registers (`t0-t6`) as needed.

3. RARS Introduction

- (a) If you have not already done so for lecture, download the RARS simulator from your Canvas course (the version provided is custom-rolled for this course). Read the ‘RARS’ tab information under the help menu (question mark icon or under ‘Help’ → ‘Help’). Some of the specifics of RISC-V that are referenced you may just be learning about or haven’t learned about at all in the case of ecalls/syscalls (or will never learn in 3810 in the case of MIDI tools). You should, however, be able to answer: does RARS simulate the ABI, ISA, and/or microarchitecture/implementation of a RISC-V processor? Explain. Where would you look for error messages? Where would you look for text output from a running program? Where would you enter text as input? Where would you be able to find the contents of memory location 0x10010004? *[We’ll use this version of RARS since it is used for testing the RISC-V processors you will design for your term project.]*
- (b) In RARS, load prob3.s into your simulator. Run three times with different inputs and report the result (and corresponding inputs used). Looking at the code, what does this program do? Include both English sentences describing its functionality and some C code that *could* compile to this. Submit your C code as a compilable file prob3_<NetID>.c.
- (c) Modify prob3.s to find the minimum of two array values and write the value back to a third array location. The minimum of two numbers, x and y, can be given by: $\text{MIN}(x, y) = (x + y - \text{ABS}(x - y))/2$. Note that ABS is not a RISC-V instruction, but can be implemented using only arithmetic, logical, and shift instructions.
All three indices should be entered by the user. For this assignment you may assume that the user inputs correct values. Provide three test cases (inputs and observed/expected outputs) showing your code works and describe how you verified correctness. Submit your new program as prob3_<NetID>.s.